

Lethabo Ngobeni

NGBLET008

CSC2009F

Report: Place Search Experiments

1 Object-Oriented Design

Classes and Responsibilities:

1. PlaceNameEntry
 - Represents a single place with its name and optional metadata (province, coordinates, etc.).
 - Methods: `getPlaceName()`, `setPlaceName()`, `toString()`.
2. PlaceNameArray
 - Stores an array of PlaceNameEntry objects. Handles addition, duplicate checks, and linear search.
 - Attributes: `data[]`, `size`, `comparisons`.
 - Methods: `add(entry)`, `search(name)`, `getSize()`, `getComparisons()`.
3. PlaceNameBST
 - Stores PlaceNameEntry objects in a binary search tree. Supports insertion and search.
 - Methods: `insert(entry)`, `search(name)`, `getHeight()`, `getComparisons()`.
4. QueryGenerator
 - Generates random or sample queries for testing search algorithms.
 - Method: `generateQueries(fileName, numQueries): String[]`.
5. Experiment
 - Runs performance tests for different data structures and records metrics (comparisons, time).
 - Attributes: CSV input file, list of N values, queries.
 - Methods: `runExperiment()`, `saveResults()`.

Interactions:

- Experiment loads data and populates PlaceNameArray or PlaceNameBST with PlaceNameEntry objects.
- QueryGenerator provides search queries for Experiment.
- Data structures (Array or BST) execute searches and return metrics for analysis.

2 Interactive Application Testing

Sample Output:

Loading data from SAPlaceNames.csv... 1000 entries loaded.

Successful search:

```
--- Place Search Array Menu ---
1. Load dataset
2. Search for a place name
3. Quit
Enter your choice: 2
Enter place name to search: Cape Town
Found: Cape Town, City of Cape Town, Western Cape, 433688
Comparisons made: 330
```

Unsuccessful search:

```
--- Place Search Array Menu ---
1. Load dataset
2. Search for a place name
3. Quit
Enter your choice: 2
Enter place name to search: America
Place name not found.
Comparisons made: 993
```

Edge Cases Tested:

- Searching for the first and last entries in the array/BST.
- Searching for non-existent place names.
- Empty input file (array/BST handles gracefully without crashing).
- Duplicate entries in input file (array/BST prevents duplicates).

3. Experimental Results and Discussion

Results Table :

N	Array	BST-Random	BST-Sorted	BST-Optimal
1000	945,63	13,01	488,10	9,92
2000	1785,44	14,40	894,56	10,86
6000	4185,36	16,36	2461,52	12,02
7000	4679,17	16,67	2909,60	12,17
8000	5168,87	17,06	3355,78	12,38
9000	5542,27	17,30	3805,30	12,51
10000	5827,99	17,41	4229,67	12,57

Graph:

Discussion Points:

- Array: Linear search grows linearly with N, matches theoretical $N/2$.
- BST As-Is: Random insertion approximates $\log_2 N$ growth, slightly above theoretical due to uneven distribution.
- BST Sorted: Degenerates to a linked list; search grows linearly (matches array).
- BST Optimal: Balanced insertion ensures minimal height; search grows $\log_2 N$.
- Insertion order effect: Shows importance of BST balancing; sorted input leads to worst-case performance.
- Results match theoretical predictions and validate lecture models.

AI Usage Disclosure

For this assignment, AI assistance (ChatGPT, GPT-5) was used in the following ways:

1. **Clarifying programming concepts** – Understanding object-oriented design, classes, arrays, and algorithms relevant to the assignment.
2. **Code guidance and debugging** – Helping identify errors in Java code (e.g. exception handling, array operations) and suggesting corrections.
3. **Code structure advice** – Suggestions on organizing classes, methods, and interactions for clarity and maintainability.
4. **Report guidance** – Assistance with explaining class diagrams, object-oriented design sections, and experimental results in clear written form.
5. **Formatting and presentation** – Recommendations for structuring the report sections according to assignment requirements.

All final code, analysis, and submissions are the author's own work. AI was used only as a **support tool** and not as a replacement for personal work or understanding.